



Unidad 5

Patrones estructurales: cómo se arman objetos con objetos

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026



Varios objetos que se usan como uno solo.

Los estructurales combinan clases y objetos por composición, sin complicar el código que los usa.

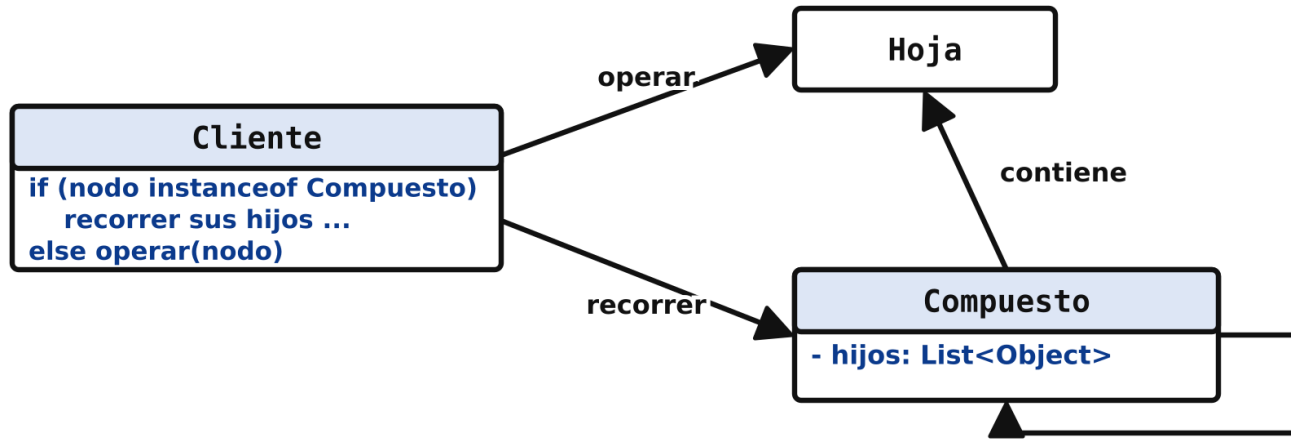


Composición antes que herencia

- La herencia decide en tiempo de compilación, de a una clase por vez.
- La composición decide en tiempo de ejecución, se puede apilar y se puede deshacer.
- **Composite:** tratar un objeto y un grupo de objetos de la misma manera.
- **Decorator:** agregar responsabilidades a un objeto sin cambiar su clase.
- **Facade:** una entrada simple a un conjunto de clases complicado.

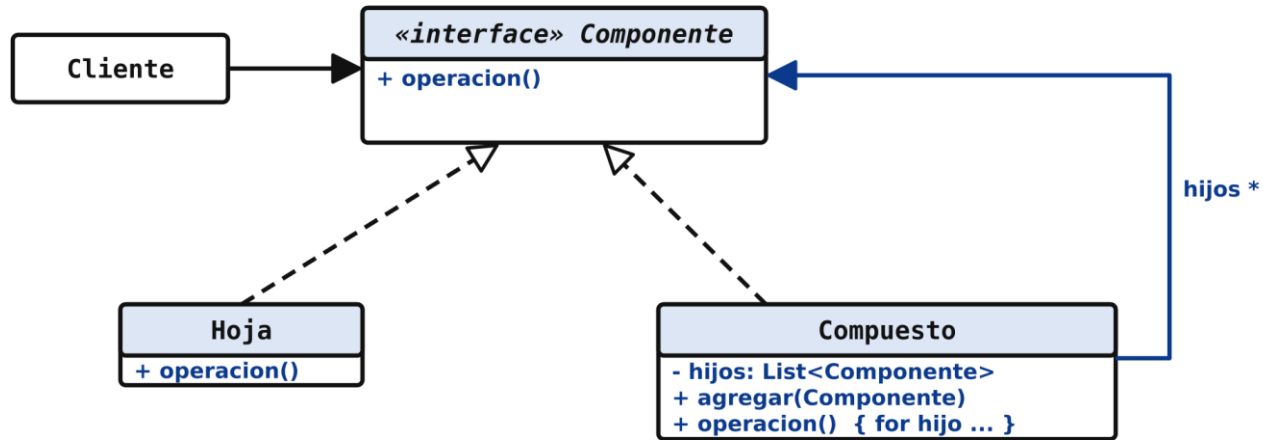
El consejo más repetido del libro de Gang of Four, aplicado de tres maneras.

Composite: el problema



El cliente pregunta si el nodo es simple o compuesto en cada operación, y recorre el árbol por su cuenta.

Composite: la solución



Hoja y Compuesto implementan la misma interfaz. El Compuesto delega en sus hijos, que son Componentes.

Composite: una carpeta es un elemento

```
public class Carpeta implements Elemento {
    private final List<Elemento> hijos = new ArrayList<>();

    public boolean agregar(Elemento e) {
        if (e == null || e == this) return false;
        return hijos.add(e);
    }

    public long tamanio() {
        long total = 0;
        for (Elemento hijo : hijos) total += hijo.tamanio();
        return total;
    }
}

documentos.tamanio(); // el pdf y la carpeta de fotos
```

`tamanio()` no sabe si cada hijo es un archivo o una carpeta con mil archivos. La recursión sale sola.

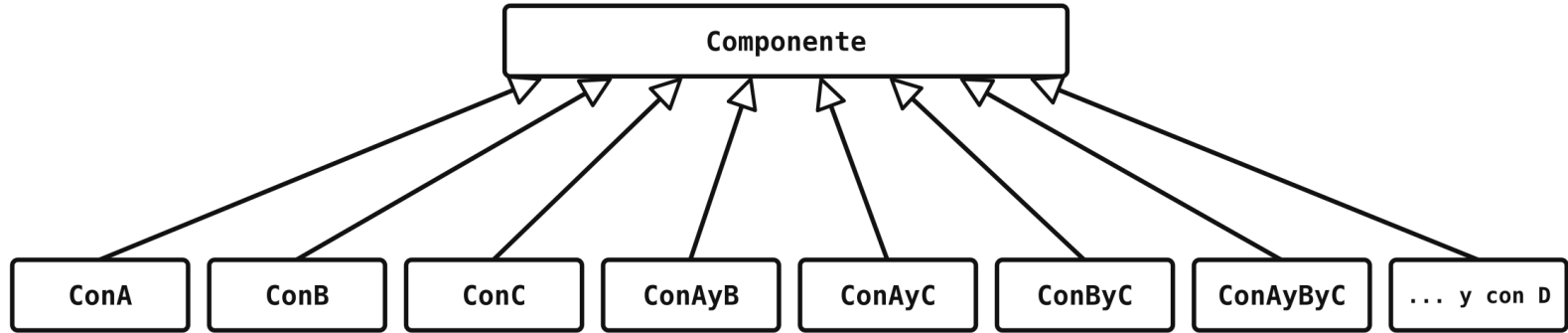


Composite: dónde va agregar()

- **En la interfaz:** el cliente trata todo igual, pero una hoja tiene que responder a `agregar()`, y solo puede lanzar una excepción.
- **Solo en el Compuesto:** es más seguro, pero el cliente tiene que saber que tiene una carpeta para agregarle algo.
- En este curso, la segunda. Una hoja que lanza "no soportado" promete lo que no cumple, y eso es romper Liskov.

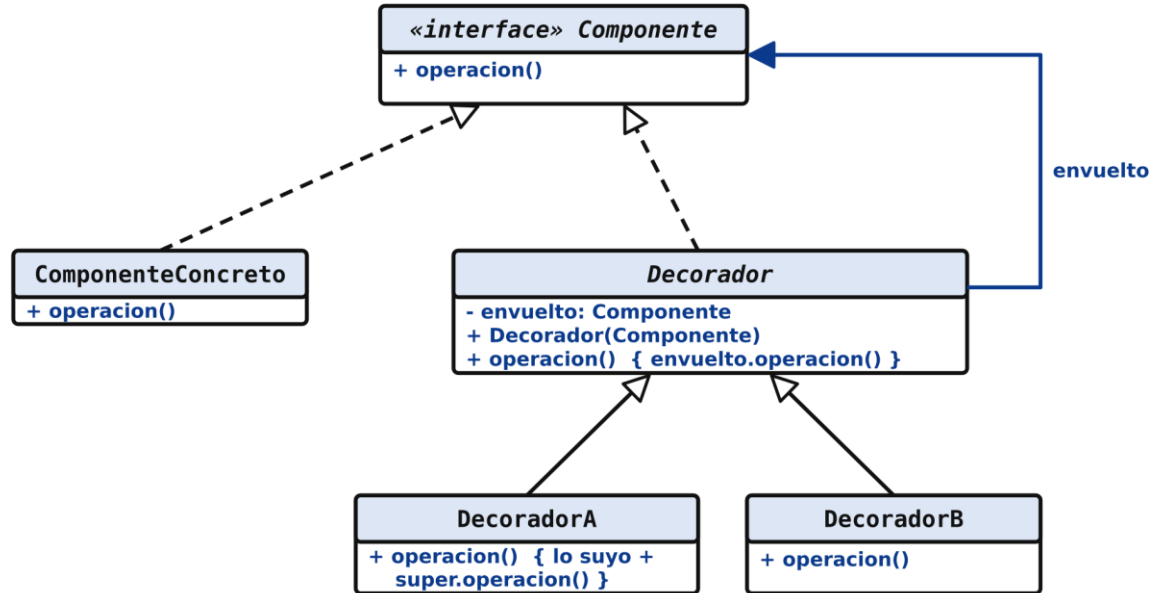
Uniformidad o seguridad. El libro original elige la primera; nosotros, la segunda.

Decorator: el problema



Una subclase por combinación, elegida al hacer el `new` y fija desde entonces.

Decorator: la solución



El Decorador implementa la misma interfaz que envuelve, así que se apila sobre el objeto base o sobre otro decorador.



Decorator: equipar es envolver

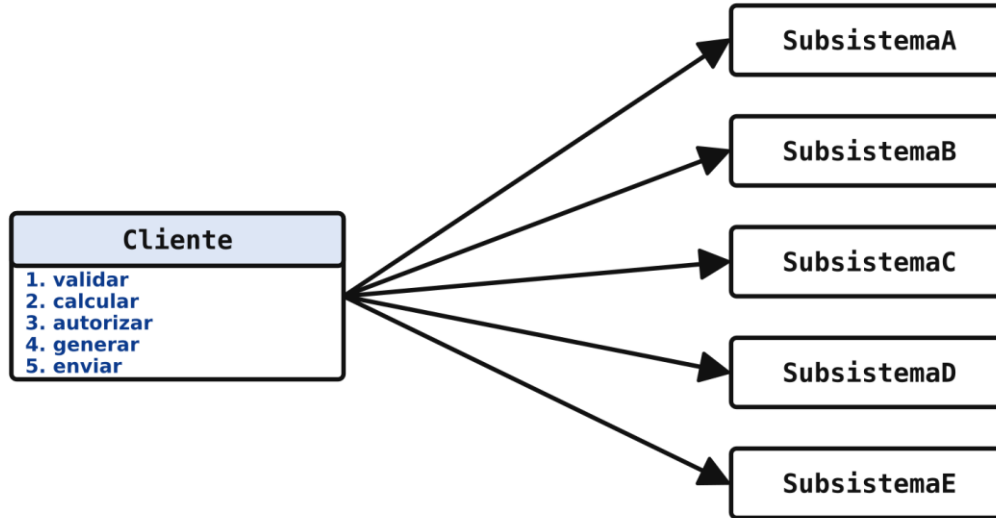
```
public abstract class Equipamiento implements Personaje {
    private final Personaje base;
    public int ataque() { return base.ataque(); } // delega
}

public class Espada extends Equipamiento {
    public int ataque() { return super.ataque() + 8; }
}

Personaje heroe = new Guerrero();
heroe = new Espada(heroe);
heroe = new Escudo(heroe);
heroe.ataque(); // 10 + 8 = 18
```

Ninguna clase se llama GuerreroConEspadaYEScudo. Perder el escudo es volver a la referencia anterior.

Facade: el problema



El cliente conoce cinco clases, el orden de las llamadas y qué dato pasa de una a otra.



Facade: una operación por llamada

```
public boolean emitir(Pedido pedido) {  
    if (!validador.esValido(pedido.getCuit()))  
        return false;  
    double total = impuestos.totalConImpuestos(pedido);  
    String codigo = autorizacion.solicitar(  
        pedido.getCuit(), total);  
    if (codigo == null) return false;  
    byte[] archivo = pdf.generar(pedido, total, codigo);  
    return correo.enviar(pedido.getCorreoCliente(), archivo);  
}  
  
// quien vende:  
facturacion.emitir(pedido);
```

Las cinco clases entran por constructor. La secuencia está escrita una vez.



Facade y MVC

- Es el patrón más simple del catálogo, y el más usado sin nombrarlo.
- En MVC, el modelo suele exponer una clase así: la partida, la caja, la cuenta.
- El controlador le pide operaciones enteras, en vez de coordinar cinco clases del modelo desde afuera.
- Cuidado: una fachada que crece sin criterio termina siendo una clase que sabe de todo.

En el proyecto, la partida es la fachada del modelo.



En la biblioteca

- **Composite:** un `JPanel` adentro de otro `JPanel`. Cuando la ventana se dibuja, cada contenedor dibuja a sus hijos.
- **Decorator:** un `JScrollPane` alrededor de una `JTable`, y `new BufferedReader(new FileReader(archivo))`.
- **Facade:** `Files.readString(ruta)`, que abre, lee, decodifica y cierra en una llamada.

Ya los usaron los tres, en la interfaz gráfica.



La actividad de hoy

- Una batalla en consola: la escuadra no es un **Combatiente**, el equipamiento son subclases **SoldadoCon...**, y cada ataque son cuatro pasos a mano.
- **Parte 1:** leer el **instanceof**, intentar meter una escuadra en otra, contar clases para una armadura.
- **Parte 2:** Composite para la escuadra, Decorator para el equipamiento, Facade para el combate.
- **Parte 3:** un estandarte para toda la escuadra de Rohan, sin tocar ninguna clase.